

Why this one prediction?

Global importance (deck 2) says temp and yr matter *overall*. But the model predicts **this particular day** at **4311 rentals** (below the ~ 4500 average) — which features drove *that*?

Global tools can't answer per-prediction. **SHAP** can: it explains a **single prediction**, and the same idea rolls up into global plots.

Still the bike-sharing random forest from decks 1–2.

Outline

What SHAP is

The SHAP plots (how to read them)

SHAP in practice

LIME

Counterfactual explanations

What SHAP is

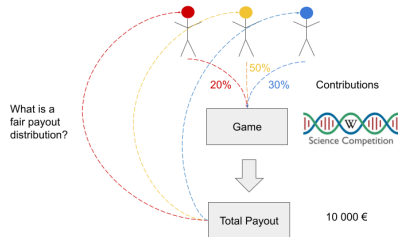
A fair split of the prediction among the features.

The idea: a fair payout

Treat the prediction as a **payout to split fairly** among the features — like a team bonus among players who contributed differently.

SHAP gives each feature its **fair share**: its contribution *averaged over all the orders* in which features could be added to the model.

That averaging is the whole game-theory part — we'll take it on trust and read the results.



Source: LMU IML (CC BY 4.0).

One prediction = base value + the SHAP values

Every SHAP explanation is one simple sum:

$$\underbrace{\hat{f}(\mathbf{x})}_{\text{this prediction}} = \underbrace{\mathbb{E}[\hat{f}(X)]}_{\text{base value (avg. prediction)}} + \sum_j \phi_j,$$

where ϕ_j is feature j 's SHAP value.

- ▶ $\phi_j > 0$ pushes this prediction **up**; $\phi_j < 0$ pushes it **down**.
- ▶ **Every SHAP plot visualizes this sum** — for one row, or aggregated over many.

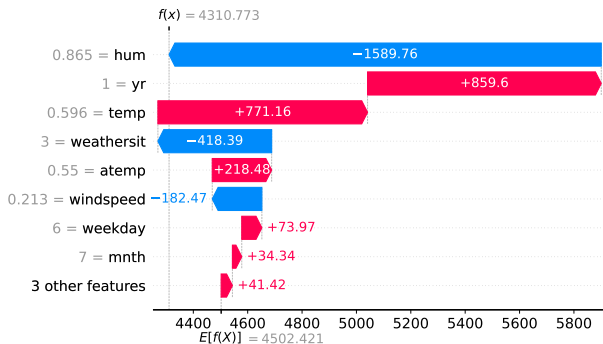
The SHAP plots — how to read them

One local plot, three global ones. Same SHAP values, different views.

Waterfall: one prediction, fully explained

Start at the **base value** (~ 4502 rentals) and add each feature's SHAP value until you reach **this day's prediction** (4311).

Read it: yr (+860) and temp (+771) push it **up**, but a very humid day (hum -1590) and poor weathersit (-418) push it **down** — landing just below average.

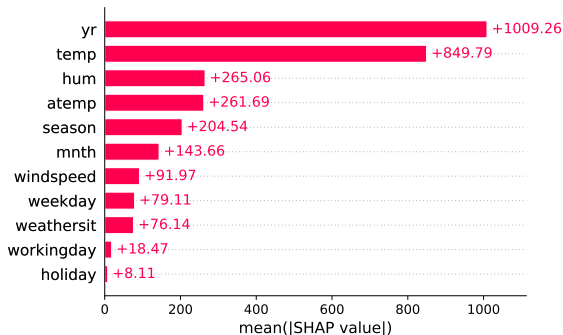


Bar: global importance (mean |SHAP|)

Average the **magnitude** of each feature's SHAP over all rows:

$$\text{importance}_j = \frac{1}{n} \sum_i |\phi_j^{(i)}|$$

- ▶ One bar per feature — **reads like the PFI bar** from deck 2.
- ▶ yr, temp dominate — roughly agreeing with PFI.



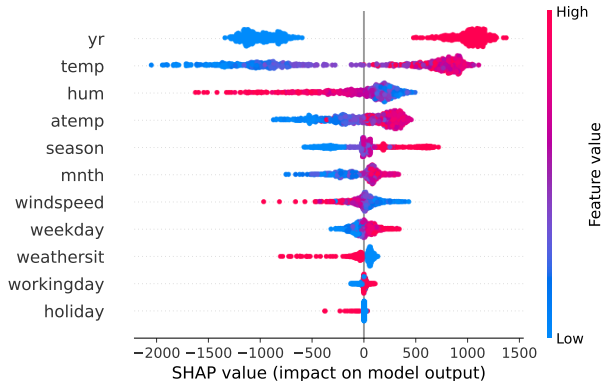
Beeswarm: the signature SHAP plot

One dot per row, per feature.

x = SHAP value; **colour** = feature value.

Read it:

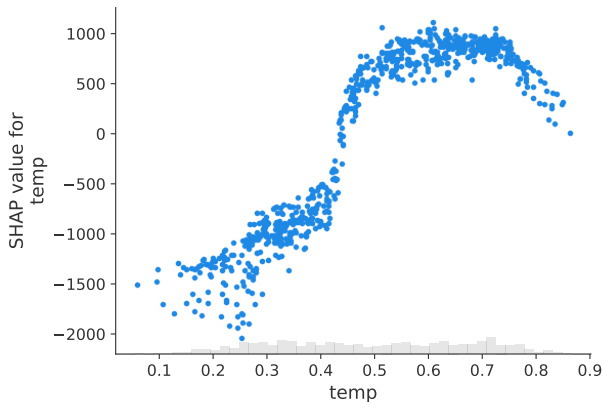
- **Spread** = how much the feature matters.
- **Colour pattern** = direction: high temp (red) → positive SHAP (more rentals); low (blue) → negative.



Dependence: the effect shape (& interactions)

SHAP value of a feature **vs. its value** — one dot per row.

- Shows the **effect shape**: rentals rise with temp, then plateau ($\approx$ a PDP).
- Colour by a second feature reveals **interactions**.
- Per-instance — *like ICE did for PDP*, it shows the spread the average hides.



TreeSHAP, and how SHAP unifies the chapter

- ▶ **TreeSHAP** computes SHAP **exactly and fast** for trees/boosting $\Rightarrow$ SHAP is the **default** for tabular models. (KernelSHAP works for any model, but is slow.)
- ▶ **SHAP unifies what we've seen:**
 - ▶ the **bar** $\approx$ PFI (global importance),
 - ▶ the **dependence** plot $\approx$ PDP (effect shape),
 - ▶ **plus** a *local* view (the waterfall) the others can't give — with a consistency guarantee (use a feature more $\Rightarrow$ its SHAP can't fall).

But like every method here, correlation muddies SHAP too — the credit split between temp and atemp stays ambiguous. Still **model behaviour, not causation** (deck 2's ice cream), and rich plots are **easy to over-read**.

SHAP in code

```
import shap
explainer = shap.TreeExplainer(model)           # fast & exact for trees/  
                                                boosting  
sv = explainer(X)                               # SHAP values for every  
                                                row  
  
shap.plots.waterfall(sv[i])                     # one prediction  
shap.plots.bar(sv)                             # global: mean |SHAP|  
shap.plots.beeswarm(sv)                        # global: dot per row,  
                                                colour = value  
shap.plots.scatter(sv[:, "temp"])              # dependence: effect  
                                                shape
```

shap.Explainer(model, X) auto-picks the right method (TreeSHAP here).

LIME: the lighter local cousin

A simple, model-agnostic local explanation — with sharper edges.

LIME: fit a simple model, locally

LIME explains one prediction with a **local surrogate**: jitter the inputs around the instance, weight by **closeness**, and fit a **simple sparse linear model** — its weights say “these few features pushed this prediction.” Works on tabular, text, & images.

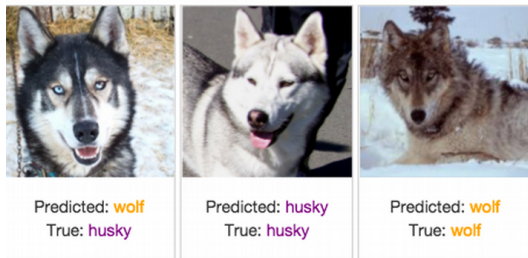
Predict-first: a husky-vs-wolf classifier is $\sim 90\%$ accurate. What did LIME reveal it actually keys on?

LIME: fit a simple model, locally

LIME explains one prediction with a **local surrogate**: jitter the inputs around the instance, weight by **closeness**, and fit a **simple sparse linear model** — its weights say “these few features pushed this prediction.” Works on tabular, text, & images.

Predict-first: a husky-vs-wolf classifier is $\sim 90\%$ accurate. What did LIME reveal it actually keys on?

The snow in the background — not the animal. A clever-Hans model, caught by a local explanation.



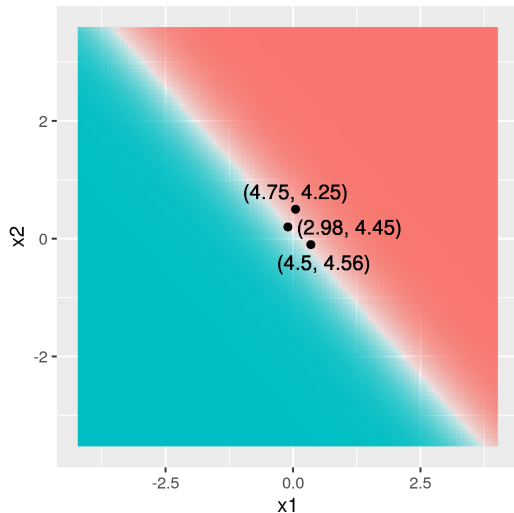
Husky vs. wolf: LIME highlights snow pixels. LMU IML (CC BY 4.0).

LIME caveats, and SHAP vs. LIME

Caveats

- ▶ **Unstable**: re-run and the explanation can change.
- ▶ The **neighbourhood** (how far to jitter, how to weight) is an *arbitrary* choice that moves the answer.

SHAP vs. LIME: SHAP has theory (consistency) + richer plots, but is slower; LIME is simpler/faster but less stable. **SHAP is the modern default.**



Counterfactual explanations

Not *why* the model decided — but *what would change* the outcome.

The smallest change that flips the prediction

SHAP and LIME answer “*why* this prediction?” A **counterfactual** answers a different, often more useful question:

What is the smallest change to the input that would flip the prediction?

- ▶ “Flip” needs a **decision**, so we switch from the (regression) bike model to the **ch. 3 bank-marketing** classifier — *will this client subscribe?* (realistic model, the leaky duration feature removed).
- ▶ A counterfactual is **actionable**: it tells you *what to change*, not just what mattered.

Example: a client predicted **won't subscribe** — the counterfactual says that *had the bank called in a different month*, the model would flip to **will subscribe**.

What makes a good counterfactual?

Five things we want at once:

- ▶ **Validity** — it actually flips the prediction.
- ▶ **Proximity** — close to the original (a small change).
- ▶ **Sparsity** — few features changed (easy to act on, easy to explain).
- ▶ **Plausibility** — stays in realistic data, no impossible rows (the extrapolation trap, deck 2).
- ▶ **Actionability** — change only what the user controls (raise a balance, not lower an age; never a protected attribute). The *recourse* angle.

Usually **several** valid counterfactuals exist — prefer the sparse, actionable ones.

Real DiCE counterfactual (bank): age, job, poutcome **frozen**; only bank-controllable levers may move.

feature	orig.	CF
age	29	29
job	mgmt	mgmt
balance	1431	1431
contact	cell.	cell.
month	jun	apr
campaign	4	4
poutcome	unkn.	unkn.
predict	no (0.44)	yes (0.78)

One change — *call in April, not June* — flips it.

Counterfactuals in code

```
import dice_ml
d    = dice_ml.Data(dataframe=df, continuous_features=num,
                    outcome_name="y")
m    = dice_ml.Model(model=clf, backend="sklearn")    # any fitted
                    classifier
exp  = dice_ml.Dice(d, m, method="random")

cf = exp.generate_counterfactuals(
    client, total_CFs=3, desired_class="opposite",
    features_to_vary=["contact", "month", "campaign"]) #
                    actionable only
cf.visualize_as_dataframe()
```

Libraries: **DiCE** (diverse counterfactuals), **Alibi**. Constrain `features_to_vary` so the advice stays *actionable*.

Same data, four rankings

One bike dataset, (mostly) one model — four “importance” methods, four different top-3s:

Method	Top 3 features
Linear coefficients (deck 1)	yr, atemp, season
Tree impurity (deck 1)	temp, yr, atemp
Permutation / PFI (deck 2)	yr, atemp, temp
SHAP (deck 3)	yr, temp, hum

They mostly agree on yr/temperature, but disagree on the rest (season? hum? temp vs atemp?). **That** is what “importance is method-dependent” means — always say *which* importance, on *what* data.

Which method when? (the whole chapter)

You want. . .	Reach for. . .
to read the model itself	a glass-box model — coefficients / a tree / rules (deck 1)
global “which features matter”	PFI or SHAP bar (decks 2–3)
“how does a feature act”	PDP / ICE or SHAP dependence (decks 2–3)
“why this one prediction”	SHAP waterfall or LIME (deck 3)
“what would change the outcome”	counterfactuals — DiCE / Alibi (deck 3)

Whatever you pick: **(1)** correlated features mislead them all, **(2)** different methods rank differently (previous slide), **(3)** none of it is *causation*.

Recap

- ▶ **SHAP** fairly splits a prediction among features: $\hat{f}(\mathbf{x}) = \text{base} + \sum_j \phi_j$.
- ▶ Read the four plots: **waterfall** (one prediction), **bar** (global importance), **beeswarm** (global, colour = value), **dependence** (effect shape + interaction).
- ▶ **TreeSHAP** makes it the default for trees/boosting; SHAP **unifies** PFI & PDP and adds a local view.
- ▶ **LIME** is the simpler local cousin — handy but unstable.
- ▶ **Counterfactuals** answer “what would change the outcome” — the smallest **valid, sparse, actionable** edit that flips the prediction (DiCE).
- ▶ Everything explains the **model**, not causation — and plots are easy to over-read.

That's interpretability: read the model when you can, explain it when you can't — importance, effects, and per-prediction attributions, each answering a different question.